Focus — Python sul tuo computer

Facoltativo. Il corso si fa sulla VM d'Ateneo, dove Python è già installato e configurato allo stesso modo per tutti. Nulla di quello che trovi qui ti serve per seguire le lezioni, fare gli esercizi o dare l'esame.

Serve se vuoi lavorare anche sul tuo portatile: in treno, senza rete, o perché ti trovi meglio. Da qui in avanti sei nel tuo ambiente, e in aula l'aiuto resta sulla VM.

---

1. Prima di installare: forse ce l'hai già

Python è preinstallato su macOS e su quasi tutte le distribuzioni Linux. Su Windows di norma no.

Apri un terminale e chiedi:

```
python3 --version
```

Se risponde qualcosa come `Python 3.12.4`, ce l'hai. Due cose da sapere:

- **per la prima parte del corso la versione conta poco**, purché sia 3: tutto quello che si fa fino a dicembre — variabili, selezione, cicli, strutture dati, funzioni, file — funziona su qualunque Python 3. Se vedi `Python 2.7`, quello non va: cerca `python3`, che è un comando diverso;
- **per pandas e matplotlib invece serve una versione recente.** Oggi pandas e matplotlib richiedono **Python 3.11 o superiore**, e numpy 3.12. Se hai un Python più vecchio, `pip install pandas` non fallisce: ti installa in silenzio una versione vecchia di pandas, e poi qualche esempio del corso non funziona senza un motivo apparente. Quel requisito cresce ogni anno, quindi il modo di saperlo è leggere l'errore o l'avviso che `pip` stampa, non fidarsi di questo numero;
- **non è la stessa versione della VM**, e non è un problema. Il codice del corso non dipende da quale 3 stai usando.

Per sapere *quale* Python stai usando, non solo quale versione:

```
which python3
```

Tieni da parte quella risposta: serve nella sezione 4.

Su Windows

Il modo meno doloroso è lo store: cerca «Python» nel Microsoft Store e installa la versione più recente. L'alternativa è python.org; in quel caso, durante l'installazione, **spunta «Add python.exe to PATH»**, altrimenti il terminale non lo troverà e sembrerà che non sia installato.

Su Windows il comando spesso è `python` e non `python3`. Provali entrambi.

---

2. Perché esistono gli ambienti virtuali

Questa sezione è il motivo per cui questo documento è lungo più di dieci righe. Salta il resto, se vuoi, ma leggi questa.

Un progetto ha bisogno di certe librerie, a certe versioni. Un altro progetto ne ha bisogno di altre, magari della stessa libreria a una versione diversa. Se installi tutto nello stesso posto, prima o poi due progetti si contendono la stessa libreria e uno dei due smette di funzionare — tipicamente quello che non stai guardando.

Un **ambiente virtuale** è una cartella che contiene un Python e le sue librerie, separata da tutto il resto. Ne crei uno per progetto, ci installi quello che serve, e quando il progetto è finito la cancelli.

Non è un'astrazione: è letteralmente una cartella.

---

3. La strada principale: `venv` e `pip`

Sono inclusi in Python, non si installa niente. Sono anche quello che troverai in qualunque corso, libro o risposta online.

Creare l'ambiente

Dalla cartella del tuo progetto:

```
python3 -m venv .venv
```

Hai creato una cartella `.venv` che contiene:

```
bin  include  lib  pyvenv.cfg
```

Il nome `.venv` è una convenzione: il punto davanti la rende nascosta, e tutti gli strumenti si aspettano di trovarla lì.

Attivarlo

Su macOS e Linux:

```
source .venv/bin/activate
```

Su Windows, in PowerShell:

```
.venv\Scripts\Activate.ps1
```

Il prompt cambia e mostra `(.venv)` davanti: è il modo in cui il terminale ti dice in quale ambiente stai. **Se non vedi quel prefisso, non è attivo**, e tutto quello che installi finisce da un'altra parte.

Vedere cosa c'è dentro

```
pip list
```

Appena creato, un ambiente contiene **una cosa sola**:

```
Package Version
------- -------
pip     26.2.1
```

Nient'altro. Non vede le librerie che hai installato fuori: è questo che significa «isolato».

Installare

```
pip install pandas matplotlib
```

`pip` scarica anche le dipendenze, quindi non stupirti se compaiono pacchetti che non hai chiesto:

```
Package         Version
--------------- -----------
numpy           2.5.3
pandas          3.0.6
pip             26.2.1
python-dateutil 2.9.0.post0
six             1.17.0
```

`numpy` è arrivato perché pandas lo richiede, non per sbaglio.

Uscirne

```
deactivate
```

Rifarlo altrove

Se vuoi che un altro computer abbia lo stesso ambiente:

```
pip freeze > requirements.txt
```

e sull'altra macchina, dentro un ambiente nuovo:

```
pip install -r requirements.txt
```

`requirements.txt` è un file di testo con i nomi e le versioni. Si mette sotto controllo di versione; la cartella `.venv` **no**, va in `.gitignore`.

---

4. Come non rompere il Python di sistema

Il tuo sistema operativo usa Python per cose sue. Se installi o aggiorni librerie nel Python di sistema, puoi rompere programmi che non c'entrano nulla con te — ed è un guaio noioso da riparare.

La regola è una: non installare mai niente con `pip` senza un ambiente attivo.

Come accorgersi se stai per farlo: se il prompt **non** mostra `(.venv)` o un nome fra parentesi, l'ambiente non è attivo. In quel caso `pip install` scriverebbe nel Python di sistema.

Le versioni recenti di Python su Linux e macOS si difendono da sole e rispondono con un errore tipo `externally-managed-environment`. Non è un problema da aggirare: **è il sistema che ti sta dicendo di creare un ambiente.** Se trovi online il consiglio di aggiungere `--break-system-packages`, quel nome è un avvertimento, non un'opzione.

---

5. L'alternativa: conda, e `mamba` al suo posto

`conda` fa due cose insieme — gestisce gli ambienti e installa i pacchetti — e sa installare anche librerie che non sono Python (compilatori, `gdal`, e simili). Nel mondo scientifico è molto diffuso.

Non installare Anaconda, che è la distribuzione grossa: porta migliaia di pacchetti che non userai. Se scegli questa strada, installa una distribuzione minima.

Installa miniforge, e usa `mamba`

Miniforge è `conda` minimo, già puntato a conda-forge, e porta con sé **`mamba`**: un sostituto di `conda` che accetta **gli stessi comandi** ed è molto più rapido a risolvere le dipendenze — dove `conda` può metterci minuti, `mamba` secondi.

Basta scriverlo al posto di `conda`:

```
mamba create -n miocorso python=3.12
mamba activate miocorso
mamba install pandas matplotlib
mamba deactivate
```

Se trovi una ricetta scritta con `conda`, funziona lo stesso: i due comandi convivono nella stessa installazione e lavorano sugli stessi ambienti.

> **I nomi si assomigliano e non sono la stessa cosa.** *Miniforge* è l'installazione da scaricare, e include `mamba`. *Mambaforge* era una variante separata, ora assorbita in Miniforge: se la trovi citata, cerca Miniforge. *Micromamba* è invece un eseguibile unico che funziona da solo, senza `conda` attorno: comodo per gli script e i container, meno per lavorarci ogni giorno.

Attenzione alla licenza. Dal 2024 i termini d'uso di Anaconda richiedono una licenza a pagamento per le organizzazioni sopra una certa dimensione, e le università ci rientrano. La questione riguarda i pacchetti scaricati dai canali di Anaconda, non `conda` in sé.

Il modo di non avere il problema è usare **miniforge**, che è lo stesso `conda` configurato per scaricare da **conda-forge**, un canale gestito dalla comunità e senza quei vincoli — e che porta `mamba` con sé. Si trova su GitHub cercando `conda-forge/miniforge`.

Se la installi per conto tuo sul tuo portatile la cosa è probabilmente irrilevante; se un giorno lavorerai in un'azienda o in un ente, non lo è, e vale la pena saperlo adesso.

La differenza che si nota subito: con `venv` l'ambiente è una cartella **dentro il progetto**; con `conda` è un ambiente **con un nome**, che vive altrove e richiami da qualunque cartella.

Non mescolare `mamba install` e `pip install` nello stesso ambiente se puoi evitarlo: funziona, finché non funziona più.

---

6. La strada moderna: uv

`uv` è uno strumento recente che fa il lavoro di `venv` e `pip` molto più in fretta, ed è compatibile con entrambi: legge `requirements.txt`, crea ambienti che `source .venv/bin/activate` attiva come gli altri.

```
uv venv
uv pip install pandas matplotlib
```

Se parti da zero oggi ed è il tuo computer, è una scelta ragionevole. Due avvertenze: è giovane, quindi quando cerchi aiuto online trovi molto più materiale su `pip`; e in aula nessuno lo userà, quindi sei da solo se qualcosa non torna.

Si installa con una riga, e le istruzioni aggiornate stanno sul suo sito — non le ricopio qui perché cambiano.

---

7. I notebook, da VS Code

Non serve installare Jupyter a parte: VS Code apre i notebook da solo.

1. installa l'estensione **Python** di Microsoft, e **Jupyter**;
2. crea un file che finisce in `.ipynb` e aprilo;
3. in alto a destra VS Code chiede **quale interprete** usare: scegli quello del tuo ambiente, `.venv/bin/python`;
4. la prima volta ti dirà che manca `ipykernel`. Lascia che lo installi: lo mette nel tuo ambiente, non nel sistema.

Quel passaggio 3 è dove si sbaglia. Se le celle non trovano `pandas` dopo che l'hai installato, quasi sempre il notebook sta usando un interprete diverso da quello in cui l'hai messo: ricontrolla l'angolo in alto a destra.

> **Il corso non usa i notebook per l'esame.** Sono comodi per esplorare dei dati; il codice che consegni è un file `.py`. Vale la pena saperli aprire, non farci tutto.

In breve

Vuoi…	Fai
seguire il corso	niente: usa la VM
lavorare sul tuo portatile	sezione 1, poi 3
capire perché tutto questo esiste	sezione 2
non rompere niente	sezione 4: mai `pip install` senza ambiente attivo
usare conda perché lo fa il tuo relatore	sezione 5: installa miniforge e scrivi `mamba`
andare più veloce	sezione 6
aprire un notebook	sezione 7

Se qualcosa non funziona sul tuo computer, in aula si guarda **dopo** quello che non funziona sulla VM: la VM è l'ambiente del corso, il tuo portatile è un extra.